

Help & User Guide

Browse topics · search messages · inspect consumer groups and live lag · reset offsets · produce & load-test messages

Version 0.1.0 · 2026-08-14

Contents

- Overview
- Running the app (dev, Docker, desktop executables)
- Connecting to a cluster (SSL / SASL / SASL_SSL, ca.pem upload)
- Exploring topics
- Searching messages
- Consumer groups — realtime lag and resetting offsets
- Producing & load testing (Testing tab)
- Security & troubleshooting

Overview

Kafka Explorer is a self-hosted web UI for exploring Apache Kafka clusters. It connects to your cluster with plaintext, TLS/SSL, or SASL authentication (PLAIN, SCRAM-SHA-256, SCRAM-SHA-512, OAUTHBEARER), lists topics, searches messages by key, value, time and partition, and shows consumer group state with live lag — including the ability to reset a group's committed offsets.

- Connections support TLS with a CA certificate (ca.pem upload), mTLS client certificates, JKS/PKCS#12 keystore + truststore uploads with passwords (converted to PEM), and SASL over TLS (SASL_SSL).
- Disconnect clears the saved connection from the browser and returns to the connection form.
- Message search: substring filters on key/value, time range, per-partition filter, sort by time, base64

view, JSON export.

- Consumer groups: state, members, live lag with a 5-second auto-refresh, and offset reset to earliest, latest, timestamp, or a custom offset.
- Runs from source, in Docker, or as single-file desktop executables for Windows, macOS and Linux.

Running the app

Development

- `bun install` — install dependencies
- `bun run dev` — Vite on :5173 (proxies /api) + API on :8787
- `bun run build` — production frontend build into dist/
- `bun run server:start` — serve the built app from the API on :8787

Desktop executables

`bun run build:desktop` cross-compiles the whole app into single-file executables in `bin/` (Windows x64 .exe, macOS arm64 + x64, Linux x64). Run the file and your browser opens at `http://localhost:8787` — no Node, Bun, or Docker required. Full instructions: `EXECUTABLES.md` in the repository.

Connecting to a cluster

Open the app and pick the connection type that matches your cluster: PLAINTEXT (no security), SSL (TLS only), SASL (auth without TLS), or SASL_SSL (TLS + SASL — the common choice for managed clusters such as Confluent, Aiven, or Redpanda).

Kafka Explorer
Browse topics · consumer groups · search messages

not connected Connection settings

CA CERTIFICATE (PEM) [ca.pem](#)

---BEGIN CERTIFICATE---

Upload or paste your ca.pem. Needed to trust self signed / private-CA brokers.

CLIENT CERTIFICATE (PEM) [cert.pem](#) CLIENT KEY (PEM) [key.pem](#)

---BEGIN CERTIFICATE---

---BEGIN PRIVATE KEY---

Only if the broker requires client certificates (mTLS). Private key for the client certificate above.

Use Java keystores (JKS / PKCS#12) ☐
Upload keystore/truststore files with passwords — converted to PEM automatically

Verify broker certificate ☒
Turn off to skip certificate validation — only for private clusters with untrusted certs

SASL authentication Credentials the broker requires

MECHANISM
SASL/PLAIN

USERNAME demo-user PASSWORD *****

Connection successful
Cluster demo-cluster-01 · 3 brokers · 5 topics

Test connection Save & explore topics

SASL_SSL connection form with ca.pem upload and SASL credentials

- Bootstrap brokers: one host:port per line (use the advertised listeners).
- TLS: upload your ca.pem to trust a private/self-signed CA; client certificate + key only if the broker requires mTLS.
- SASL: choose PLAIN or SCRAM-SHA-256/512 and enter username/password, or OAUTHBEARER with a bearer token.
- Test connection verifies brokers, TLS and auth before saving.
- Connection details are stored in the browser's localStorage and only sent to the local API.

The screenshot shows the 'Connect to a Kafka cluster' form in Kafka Explorer. At the top, there's a header with the 'K' logo, 'Kafka Explorer', and navigation links: 'Browse topics', 'consumer groups', and 'search messages'. On the right, there are buttons for 'not connected' and 'Connection settings'. The main form area has a title 'Connect to a Kafka cluster' and a subtitle 'Supports plaintext, TLS/SSL, and SASL (PLAIN, SCRAM-SHA-256/512, OAUTHBEARER). Credentials stay in your browser and are only sent to the local API.' Below this, there's a section for 'BOOTSTRAP BROKERS' with a text input containing 'localhost:9092' and 'kafka-1.example.com:9093'. A note says 'One host:port per line (or comma separated)'. Then, there's a 'CONNECTION TYPE' dropdown menu set to 'PLAINTEXT — no encryption, no auth'. A note below it says 'Pick the security model your cluster uses (e.g. SASL_SSL for SASL over TLS)'. Next is a 'CLIENT ID' text input with 'kafka-explorer'. A note says 'Optional identifier used when connecting to the cluster.' At the bottom, there are two buttons: 'Test connection' and 'Save & explore topics'. A footer note at the very bottom says 'Kafka Explorer - connection details are stored in your browser and used only to talk to the local API.'

Kafka Explorer

Browse topics consumer groups search messages

not connected Connection settings

Connect to a Kafka cluster

Supports plaintext, TLS/SSL, and SASL (PLAIN, SCRAM-SHA-256/512, OAUTHBEARER). Credentials stay in your browser and are only sent to the local API.

BOOTSTRAP BROKERS

localhost:9092
kafka-1.example.com:9093

One host:port per line (or comma separated).

CONNECTION TYPE

PLAINTEXT — no encryption, no auth

Pick the security model your cluster uses (e.g. SASL_SSL for SASL over TLS).

CLIENT ID

kafka-explorer

Optional identifier used when connecting to the cluster.

Test connection Save & explore topics

Kafka Explorer - connection details are stored in your browser and used only to talk to the local API.

Connection form — choose your security model

Exploring topics

The Topics page lists every topic with its partition and replication-factor counts. Filter by name, then click a topic to open it.

The screenshot shows the 'Topics' page in Kafka Explorer. At the top, there's a header with the 'K' logo, 'Kafka Explorer', and navigation links: 'Browse topics', 'consumer groups', and 'search messages'. On the right, there are buttons for 'connected', 'Disconnect', and 'Connection settings'. The main content area has a title 'Topics' and a subtitle '5 topics on this cluster'. Below this, there's a table with columns: 'TOPIC', 'PARTITIONS', 'REPLICATION', and 'TYPE'. The table lists five topics: 'orders', 'payments', 'user-events', 'inventory-updates', and '__consumer_offsets'. Each row has a 'user' button and a right arrow. Above the table, there are buttons for 'Filter topics...' and 'Refresh'. A footer note at the very bottom says 'Kafka Explorer - connection details are stored in your browser and used only to talk to the local API.'

Kafka Explorer

Browse topics consumer groups search messages

connected Disconnect Connection settings

Topics

5 topics on this cluster

Filter topics... Refresh

TOPIC	PARTITIONS	REPLICATION	TYPE
orders	6	3	user
payments	4	3	user
user-events	8	2	user
inventory-updates	3	2	user
__consumer_offsets	50	3	internal

Kafka Explorer - connection details are stored in your browser and used only to talk to the local API.

Topics list

Each topic opens with two tabs: Messages (search) and Consumer groups.

Searching messages

The Messages tab filters the topic by key/value substring, time range, partition, and limit, and sorts results by time (newest or oldest first). With no filters and newest-first, only the tail of each partition is read for fast results; filtered searches scan from the beginning (capped at 250k messages). Click a row to expand the full value and headers. Use the Text/Base64 toggle for binary payloads and Export JSON to download results.

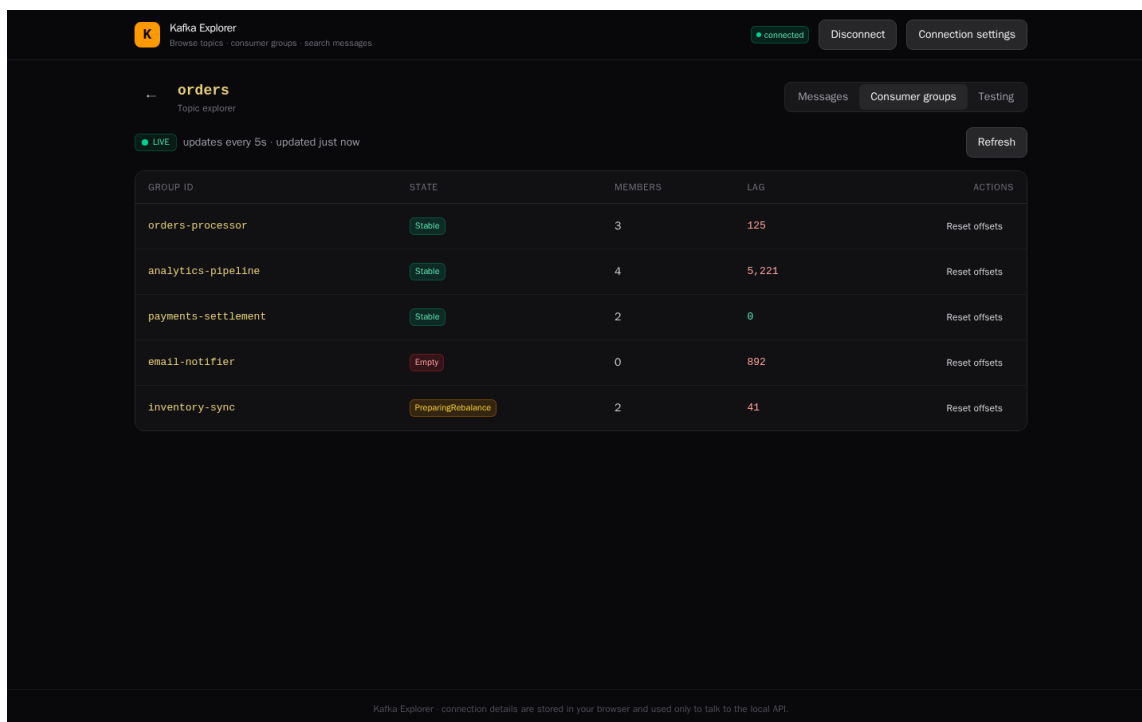
The screenshot shows the Kafka Explorer interface for the 'orders' topic. The search filters are set to: Key contains 'Substring of key', Value contains 'Substring of value', From (Local Time) 'mm/dd/yyyy, --:--', To (Local Time) 'mm/dd/yyyy, --:--', Partition 'All partitions', Limit '100 messages', and Sort by Time 'Newest first'. The search results show 12 matches, scanned 1,482,113 bytes in 612 ms. The results table has columns: TIME (UTC), PARTITION, OFFSET, KEY, VALUE, and SIZE. The values are JSON objects representing orders.

TIME (UTC)	PARTITION	OFFSET	KEY	VALUE	SIZE
2026-08-12 21:09:20.738 UTC	0	4	ord-10231	{"orderId":"ord-10231","sku":"KB-2041","qty":2,"total":129.98,"status":"paid"}	87 B
2026-08-12 21:07:20.738 UTC	1	7	ord-10232	{"orderId":"ord-10232","sku":"KB-1108","qty":1,"total":59.0,"status":"paid"}	85 B
2026-08-12 21:09:20.738 UTC	2	9	ord-10233	{"orderId":"ord-10233","sku":"KB-2041","qty":5,"total":324.95,"status":"shipped"}	90 B
2026-08-12 20:55:20.738 UTC	3	3	ord-10234	{"orderId":"ord-10234","sku":"KB-9912","qty":1,"total":899.0,"status":"paid"}	86 B
2026-08-12 20:49:20.738 UTC	4	11	ord-10235	{"orderId":"ord-10235","sku":"KB-1108","qty":3,"total":177.0,"status":"cancelled","reason":"stock"}	108 B
2026-08-12 20:40:20.738 UTC	5	2	ord-10236	{"orderId":"ord-10236","sku":"KB-2041","qty":1,"total":64.99,"status":"paid"}	86 B
2026-08-12 20:33:20.738 UTC	0	5	ord-10237	{"orderId":"ord-10237","sku":"KB-8871","qty":2,"total":239.98,"status":"shipped"}	90 B
2026-08-12 20:26:20.738 UTC	1	8	ord-10238	{"orderId":"ord-10238","sku":"KB-1108","qty":1,"total":59.0,"status":"paid"}	85 B
2026-08-12 20:18:20.738 UTC	2	10	ord-10239	{"orderId":"ord-10239","sku":"KB-9912","qty":1,"total":899.0,"status":"paid"}	90 B

Message search results with filters

Consumer groups

The Consumer groups tab shows the groups subscribed to the topic: state (Stable, PreparingRebalance, Dead, ...), member count, and lag. Lag updates every 5 seconds while the tab is open — the LIVE badge shows when data was last refreshed, so you can watch consumers drain (or fall behind) in real time.

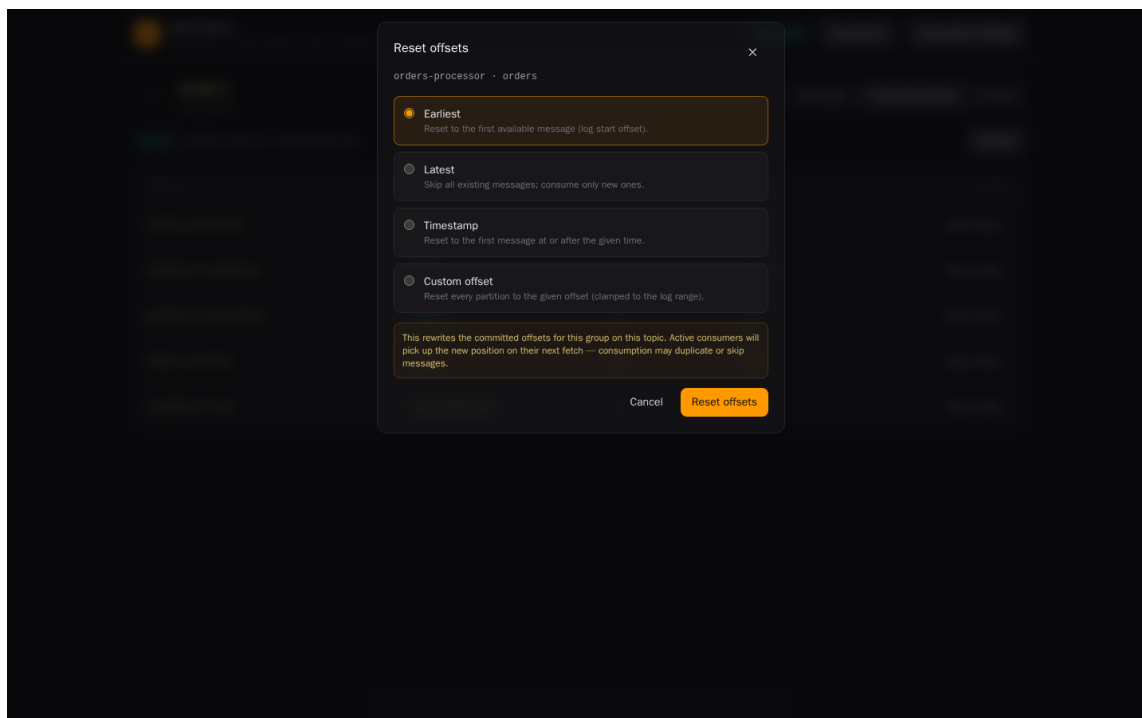


GROUP ID	STATE	MEMBERS	LAG	ACTIONS
orders-processor	Stable	3	125	Reset offsets
analytics-pipeline	Stable	4	5,221	Reset offsets
payments-settlement	Stable	2	0	Reset offsets
email-notifier	Empty	0	892	Reset offsets
inventory-sync	PreparingRebalance	2	41	Reset offsets

Consumer groups with live lag

Resetting offsets

Use Reset offsets to move a group's committed position on this topic: Earliest (start from the beginning), Latest (skip existing messages), Timestamp (first message at/after a given time), or a Custom offset (clamped to the log range). The dialog shows the previous and new offset per partition after the reset. Active consumers pick up the new position on their next fetch — consumption may duplicate or skip messages.



Reset offsets

orders-processor · orders

☒ Earliest
Reset to the first available message (log start offset).

☐ Latest
Skip all existing messages; consume only new ones.

☐ Timestamp
Reset to the first message at or after the given time.

☐ Custom offset
Reset every partition to the given offset (clamped to the log range).

This rewrites the committed offsets for this group on this topic. Active consumers will pick up the new position on their next fetch — consumption may duplicate or skip messages.

Cancel Reset offsets

Reset offsets dialog

Producing & load testing

Each topic has a Testing tab (marked WRITES) for testing your workload against the cluster. It produces messages and rewrites committed offsets, so use it with care on shared environments.

←

orders

Topic explorer

MessagesConsumer groupsTesting

WRITES

The Testing tab produces messages and rewrites offsets on topic **orders** — use with care.

Produce a message

Write one message to this topic for testing.

KEY

ord-98001

Optional. Leave empty for a null key.

PARTITION

0

Optional — default: round-robin.

HEADERS

event-type: test.message

One 'key: value' per line. Optional.

VALUE

```
{"orderId": "ord-98001", "qty": 1, "status": "paid"}
```

Leave empty to write a tombstone (null value).

Produced successfully

topic orders → partition 2, offset 101234

Produce message

K

Kafka Explorer

Browse topics · consumer groups · search messages

connectedDisconnectConnection settings

MESSAGE COUNT

100

1 – 100,000.

RATE (MSG/SEC)

e.g. 1000

Empty = as fast as possible.

PARTITION

0

Optional — default: round-robin.

KEY TEMPLATE

ord-{{i}}

Rendered per message (may be empty).

HEADERS

event-type: test.bulk

Static headers on every message. 'key: value' per line.

SAMPLE PAYLOAD TEMPLATE

```
{
  "orderId": "ord-{{i}}",
  "sku": "KB-2041",
  "qty": {{rand:10}},
  "total": {{rand:1000}}.99,
  "status": "paid",
  "userId": "user-{{i}}",
  "eventTime": "{{ts:epoch}}"
}
```

Start from your real payload and swap in placeholders.

Preview first message

Generate & post 100 messages

key: ord-0

value: {
 "orderId": "ord-0",
 "sku": "KB-2041",
 "qty": 9,
 "total": 393.99,
 "status": "paid",
 "userId": "user-0",
 "eventTime": "2026-08-14T21:21:24.168Z"
}

Produced 100 messages in 0.42s (238 msg/s)

topic: orders

Partition	First offset	Last offset	Messages
0	100000	100016	17
1	100010	100026	17
2	100020	100036	17
3	100030	100046	17
4	100040	100056	17
5	100050	100064	15

Consumer group test actions

Move a group's committed offset — handy for replaying messages or clearing lag.

CONSUMER GROUP

orders-processor (lag 127)

Groups subscribed to this topic.

CUSTOM OFFSET

e.g. 1000

Set every partition to this offset (clamped to the log range).

Reset to earliest

Clear lag (reset to latest)

Set custom offset

Reset rewrites the committed offsets for this group on this topic. Active consumers pick up the new position on their next fetch — consumption may duplicate or skip messages. This writes to your cluster.

Kafka Explorer · connection details are stored in your browser and used only to talk to the local API.

Testing tab: produce a message, generate load-test data, and reset group offsets

Produce a message

Send a single message to the topic with an optional key, partition, and headers. The result shows the partition and offset the broker assigned.

Generate & post test data (load test)

Paste a sample payload and swap in placeholders so parameters change per message: `{{i}}` (message index), `{{ts}}` / `{{ts_iso}}` (generation time), `{{uuid}}` (random UUID), `{{rand}}` / `{{rand:N}}` (random number), `{{randstr:N}}` (random string). Preview the first message before sending, then post up to 100,000 messages — optionally rate-limited — and review duration, throughput, and per-partition first/last offsets.

Consumer group test actions

Pick a group subscribed to the topic and reset it to the earliest offset, clear its lag (reset to latest), or set a custom offset (clamped to the log range) — handy for replaying messages during testing.

Security & troubleshooting

- The API is unauthenticated and binds to 0.0.0.0 — only run it on machines you trust; don't expose port 8787 to the public internet.
- Kafka credentials are stored in the browser in plain text (localStorage) — avoid production secrets in a shared browser.
- Desktop executables are not code-signed: macOS Gatekeeper (right-click !' Open) and Windows SmartScreen (More info !' Run anyway) may warn.
- Connection timeout !' broker unreachable from the machine running the app; check advertised listeners and firewall.
- Certificate error !' upload the broker CA, or disable certificate verification for private clusters.
- SASL authentication failed !' wrong mechanism or credentials; SCRAM-SHA-512 vs 256 matters.
- Port 8787 in use !' set `KAFKA_EXPLORER_API_PORT`.